

Cinema Central

Sistem de Gestionare a unui Cinematograf

Proiect realizat de: **Calancea Daniel**

Grupa: **3121B**

Limbaj: **C++17** | UI: **ANSI Terminal + Qt6**

1. Descriere Generala

Proiectul **Cinema Central** este un sistem complet de gestiune a unui cinematograful, implementat in C++17 cu doua tipuri de interfete utilizator: un UI terminal ANSI pentru personalul intern si un UI grafic Qt6 pentru clientii care rezerva online.

Aplicatia suporta trei roluri de utilizatori cu permisiuni diferite, gestioneaza filme, sali si rezervari, si include un sistem de calcul automat al preturilor in functie de tipul locului si ziua saptamanii.

Functionalitati principale:

Functionalitate	Detalii
Gestiune filme	Adaugare, stergere, cautare dupa titlu, filtrare dupa tip (2D/3D) si gen
Gestiune sali	Sali cu randuri si coloane configurabile, locuri VIP / Standard / Student
Rezervari	Rezervare fizica (angajat) si online (client), cu calcul automat al pretului
Calcul pret	Baza 2D=30 lei, 3D=45 lei; VIP +50%, Student -20%, Weekend +15%
UI Terminal	Meniu colorat ANSI pentru Administrator si Angajat
UI Qt6	Interfata grafica cu carduri de filme, harta sali si dialog de rezervare
Teste unitare	12 teste automate care verifica preturile, exceptiile si rezervarile
CI/CD	GitHub Actions: compilare si testare automata la fiecare push

2. Arhitectura Proiectului

Structura folderelor:

```
cinema-sistem/  
include/ – headere (.h): toate clasele si interfata  
src/ – implementari (.cpp) + main.cpp  
ui_terminal/ – UI ANSI pentru Admin si Angajat  
ui_qt/ – UI grafic Qt6 pentru Client  
tests/ – teste unitare  
.github/workflows/ – CI/CD GitHub Actions  
CMakeLists.txt – configurare build
```

Ierarhia claselor:

Proiectul foloseste o interfata pura **ICinemaService** implementata de clasa **Cinematograf**. Aceasta separare permite testarea si extinderea usoara a sistemului.

Clasa / Interfata	Mosteneste din	Rol
ICinemaService	—	Interfata pura (contract)
Cinematograf	ICinemaService	Logica completa de business
Film	—	Entitate film cu ID unic
Sala	—	Matrice de locuri cu tipuri
Rezervare	—	Rezervare fizica la ghiseu
RezervareOnline	Rezervare	Rezervare cu email client
CinemaException	std::exception	Exceptie de baza
LocOcupatException	CinemaException	Loc deja ocupat
FilmNotFoundException	CinemaException	Film negasit dupa ID

3. Concepte OOP Utilizate

Abstractizare

ICinemaService definește un contract prin metode pur virtuale (= 0). Codul UI lucrează cu ICinemaService& fără să cunoască implementarea concretă.

Mostenire

RezervareOnline moștenește Rezervare și adaugă emailClient și trimitConfirmare(). Excepțiile formează o ierarhie: CinemaException -> LocOcupatException etc.

Polimorfism

vectorul std::vector<Rezervare> conține atât Rezervare cât și RezervareOnline. Apelul r->afiseaza() alege automat metoda corectă prin vtable.

Encapsulare

Sala păstrează matricea de locuri privată. Accesul se face prin ocupa(), elibereaza() și esteLiber(), care validează indexii și aruncă excepții la nevoie.

Smart pointers

std::weak_ptr și std::weak_ptr gestionează memoria automat, fără delete manual. Evită memory leaks.

Excepții custom

Fiecare situație de eroare are propria clasă de excepție cu mesaj descriptiv. UI-ul prinde CinemaException& și afișează mesajul fără să se prabusească.

4. Roluri Utilizatori si Interfete

Rol	Acces	Interfata
Administrator	Afiseaza filme/sali/rezervari Adauga/sterge filme Adauga/sterge sali	Terminal ANSI (--staff -> 1)
Angajat	Afiseaza filme/sali/rezervari Creeaza rezervari fizice pentru clientii la ghiseu	Terminal ANSI (--staff -> 2)
Client	Vede filmele disponibile Selecteaza locul dorit Rezervare online cu email	UI grafic Qt6 (--client)

Calculul pretului biletului:

Pretul final se calculeaza inmultind pretul de baza cu modificatorul tipului de loc si eventual cu bonusul de weekend:

Factor	Valoare
Pret baza 2D	30 lei
Pret baza 3D	45 lei
Loc Standard	x 1.0 (fara modificare)
Loc VIP	x 1.5 (+50%)
Loc Student	x 0.8 (-20%)
Weekend (Sambata/Duminica)	x 1.15 (+15%)

5. Compilare si Rulare

Compilare manuala (Linux / WSL):

```
# Modul terminal (fara Qt)

g++ -std=c++17 -Iinclude \
src/Film.cpp src/Sala.cpp src/Rezervare.cpp src/RezervareOnline.cpp \
src/Cinematograf.cpp \
ui_terminal/UITerminal.cpp ui_terminal/UIAdmin.cpp ui_terminal/UIAngajat.cpp \
src/main.cpp -o cinema


# Rulare ca Administrator

./cinema --staff # alege [1] Administrator


# Rulare ca Angajat

./cinema --staff # alege [2] Angajat
```

Compilare cu Qt (Windows — Qt Creator):

1. Deschide **CMakeLists.txt** in Qt Creator din folderul proiectului.
2. In **Projects** → **Build Settings** → **CMake**, activeaza checkbox-ul **BUILD_QT_UI**.
3. Apasa **Run CMake** apoi **Build** → **Rebuild All**.
4. In **Projects** → **Run Settings**, seteaza **Command line arguments** la **--client**.
5. Apasa butonul verde **Run** ■.

Rulare teste unitare:

```
g++ -std=c++17 -Iinclude \
src/Film.cpp src/Sala.cpp src/Rezervare.cpp src/RezervareOnline.cpp \
src/Cinematograf.cpp tests/test_rezervari.cpp -o cinema_tests

./cinema_tests
```

6. Teste Unitare

Proiectul include 12 teste unitare implementate manual (fara framework extern), organizate in categorii:

Categorie	Test	Ce verifica
Preturi	test_pret_2D_standard	30 lei pentru loc Standard 2D
Preturi	test_pret_3D_standard	45 lei pentru loc Standard 3D
Preturi	test_pret_3D_vip	67.5 lei pentru loc VIP 3D
Preturi	test_pret_2D_student	24 lei pentru loc Student 2D
Exceptii	test_loc_ocupat_arunca_exceptie	LocOcupatException la loc deja rezervat
Exceptii	test_index_invalid_arunca_exceptie	IndexInvalidException la rand invalid
Exceptii	test_index_negativ_arunca_exceptie	IndexInvalidException la index negativ
Rezervare online	test_rezervare_online_email	Email stocat corect
Rezervare online	test_rezervare_online_confirmare	trimitConfirmare() seteaza flag
Sala	test_sala_initial_libera	Toate locurile libere la creare
Sala	test_sala_locuri_libere_scad	Numarul de locuri scade dupa rezervare
Sala	test_sala_eliberare_loc	Locul devine liber dupa anulare

Toate cele 12 teste trec cu succes. CI/CD-ul GitHub Actions ruleaza aceste teste automat la fiecare push pe branch-urile **develop** si **feature/****.

7. Workflow Git

Proiectul foloseste un workflow cu branch-uri de feature, toate mergand in **develop**:

Branch	Continut
develop	Branch principal de integrare
feature/ci-cd	GitHub Actions workflow (ci.yml)
feature/ui-qt	Interfata grafica Qt6 completa

Commit-uri principale pe develop:

- C1: Structura proiect, CMakeLists.txt si ICinemaService
- C2: Clase Film si Sala cu exceptii custom
- C3: Clase Rezervare, RezervareOnline si calcul pret
- C4: Clasa Cinematograf — logica completa backend
- C5: UI terminal Administrator
- C6: UI terminal Angajat
- C7+C8: UI Qt — ecran principal, SalaWidget si rezervare online
- C9: GitHub Actions CI/CD workflow
- C10: Teste unitare (12 teste, toate trec)

8. Concluzie

Proiectul **Cinema Central** demonstreaza aplicarea principiilor fundamentale ale programarii orientate pe obiecte in C++17:

- Interfata pura ICinemaService ca contract pentru implementari interschimbabile
- Ierarhie de exceptii custom pentru gestionarea erorilor specifice domeniului
- Polimorfism prin vectori de pointeri la clasa de baza (Rezervare / RezervareOnline)
- Encapsulare stricta in clasa Sala cu validare la nivel de metoda
- Gestionare automata a memoriei prin smart pointers (unique_ptr)
- Doua tipuri de interfete: terminal ANSI si grafic Qt6 — acelasi backend
- 12 teste unitare automate si pipeline CI/CD GitHub Actions

Repository GitHub: [usmro / 3121B-Calancea-Daniel-Cinema_2.0](#)